SD^2: Software Design Document

1. Project Overview

Introduction and Problem Statement

- "Personalized Fitness Scheduler" introduces the solution to the problem of following a regular, personalized workout schedule. Most of them fail to track workouts and progress, and they are also devoid of a systematically organized mechanism regarding managing fitness goals. A project is hereby presenting a workout scheduler that will enable users in managing or tracking their workouts to reach customized fitness goals.

Stakeholders

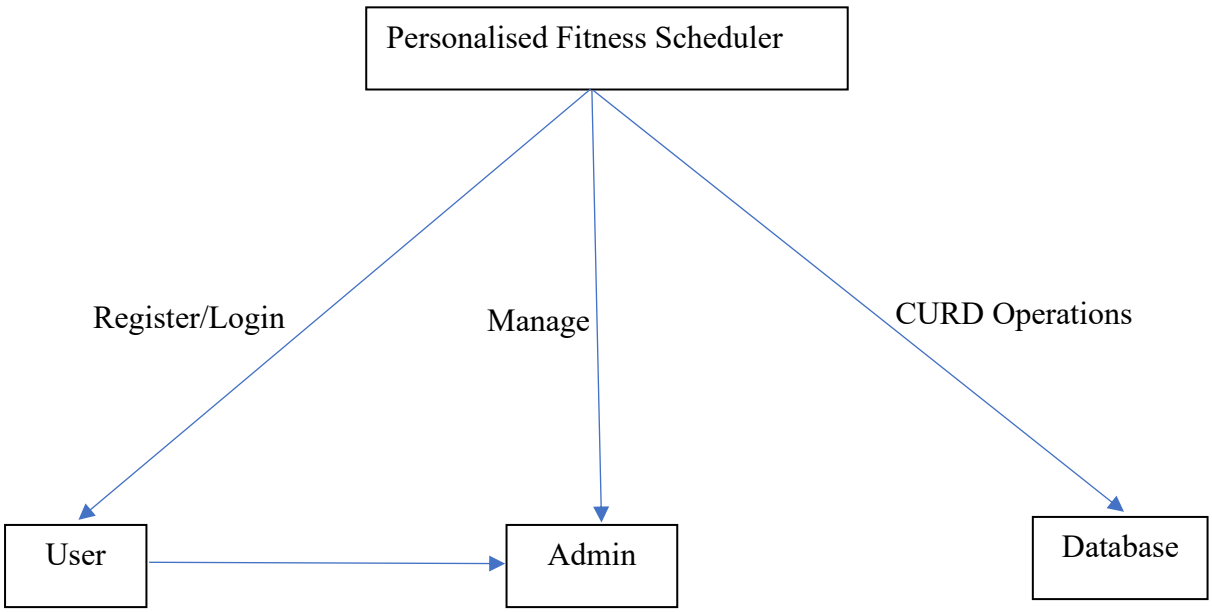
- **End-users:** Those interested in working out, who would want to schedule and log the progress about this.
- **Gym trainers:** Those people who might want to assist users by creating workout schedules and tracking some of the fitness metrics.
- **Developers:** This will be the technical people that will be involved in building, maintenance, and updating of the system.

Overview of Solution

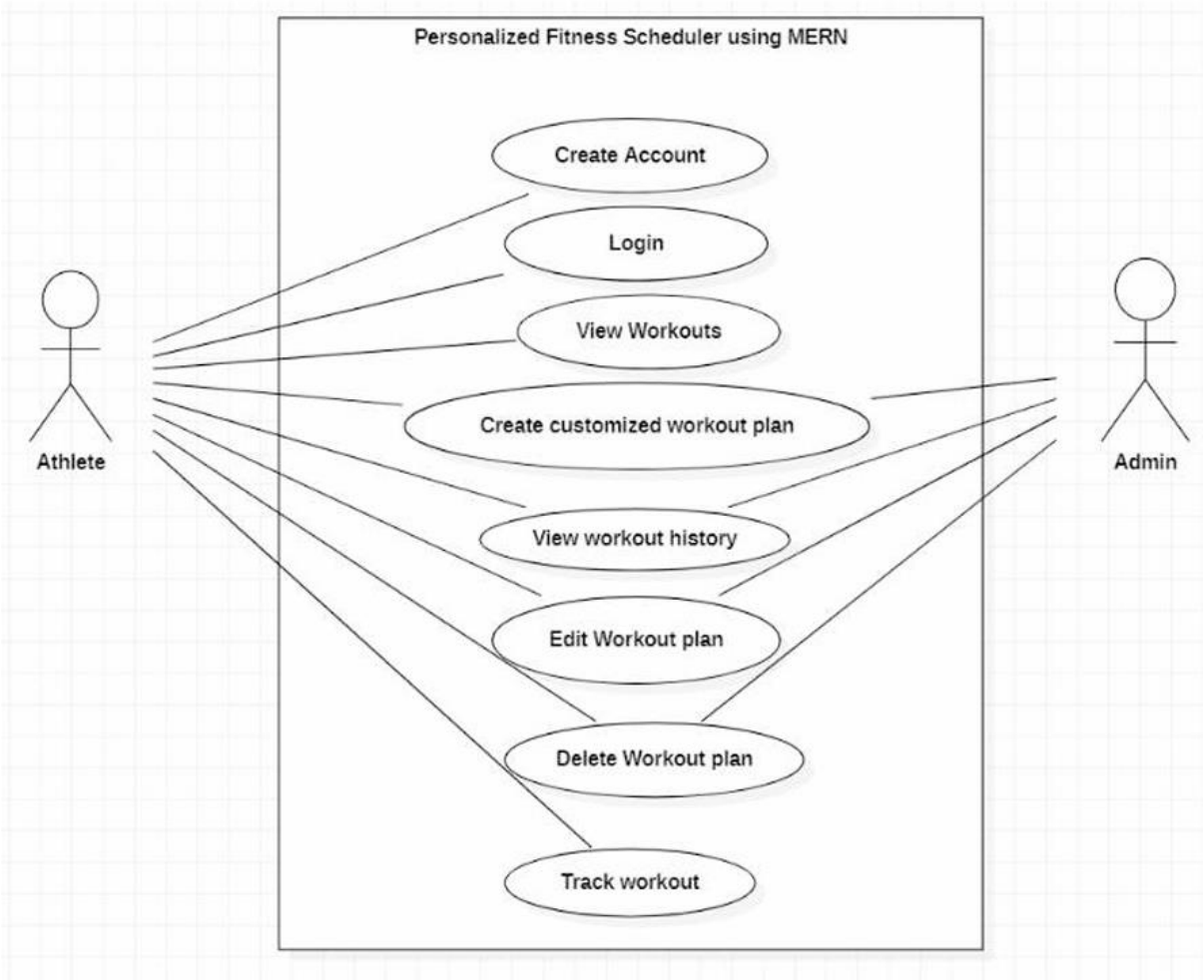
- The project is dedicated to an interactive, dynamic fitness scheduler, created on the MERN stack, including MongoDB, Express.js, React.js, and Node.js. Its key features include profile management, personalized development of workout schedules, BMI calculators, and a calendar feature of workout history.

General Model

- **Context Diagram**: It shows the user interactions with the system, including database access and interactions with the fitness scheduler module.



- **Use-case Diagram**:



  - **Create Account:** Allows a new user to sign up and create an account on the platform.
  - **Login:** Enables an existing user to access their account using valid credentials.
  - **View Workouts:** Lets users browse available workout plans or routines.
  - **Create Customized Workout Plan:** Allows users to design a personalized workout plan based on their preferences and goals.
  - **View Workout History:** Provides a history of past workouts, allowing users to track their progress over time.
  - **Edit Workout Plan:** Enables users to make adjustments to their existing workout plan.
  - **Delete Workout Plan:** Allows users to remove an existing workout plan from their profile.
  - **Track Workout:** Lets users record their workout sessions, which helps in monitoring their consistency and improvements.

- **User Stories:**
    - Users should be able to create an account by signing up so that my data relating to work-outs is saved securely, and at any time, they can access it to manage my journey of keeping fit with ease.
    - Users should be able to create a customized workout plan with regard to my fitness level and goal so that they can work out in a structured manner according to my objectives relating to health.
    - They want it to be possible as a user to view in detail my workout history, including metrics and visual charts of progress, so that they can evaluate and work on aspects of my exercise routine that need improvement.
- **Product Backlog**:
    - List prioritized features, like calendar integration, and confirmation password, in alignment with the action items from our recent TA meeting.

2. Architectural Overview

Overview of Architecture

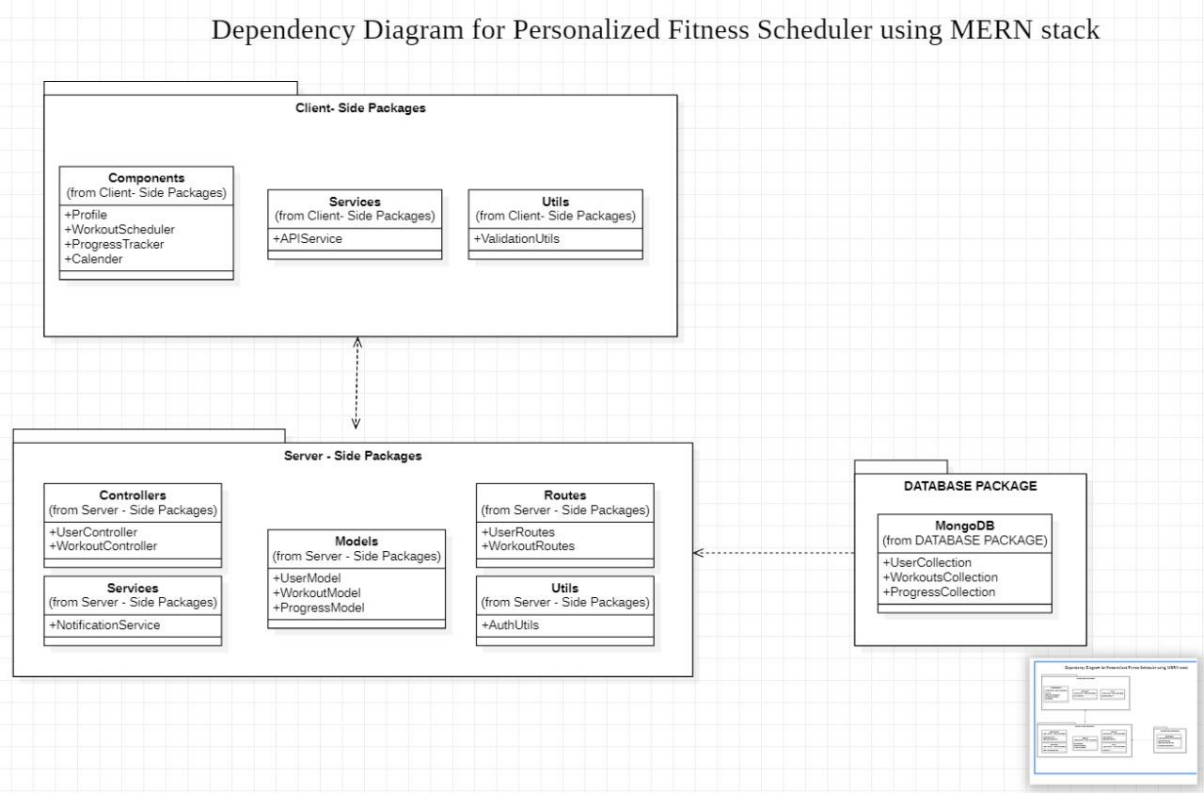
- This is a client-server architecture based on the MERN stack. React takes care of the front-end, providing the user with an intuitive interface via which they will operate the schedules. The server side will be handled through Node.js and Express to manage API calls and data processing. MongoDB serves as the database for user data, workout schedules, and profile details.

Alternative Designs Considered

- **Relational Database vs. NoSQL**: Considered using a SQL database for data consistency but chose MongoDB for flexible, scalable document storage.
- **Backend Frameworks**: Considered using Django or Flask but opted for Node.js for seamless integration with JavaScript on both client and server.

2.1 Subsystem Architecture

2.1.1 Dependency Diagram:



Client-Side Packages

Components

- **Profile**: Manages and displays user profile information, including BMI and personal details.
- **WorkoutScheduler**: Allows users to create and manage their workout schedules.
- **ProgressTracker**: Tracks user progress over time based on completed workouts and fitness goals.
- **Calendar**: Displays a calendar view for users to track workout dates and history.

Services

- **APIService**: Handles HTTP requests to communicate with the server-side APIs.

Utils

- **ValidationUtils**: Provides utilities to validate user inputs, such as email and password strength.

Server-Side Packages

Controllers

- **UserController**: Manages user-related operations, including profile management and authentication.

- **WorkoutController**: Handles operations related to workout management, such as creating, updating, and deleting workouts.

Models

- **UserModel**: Defines the schema and data structure for storing user information in MongoDB.
- **WorkoutModel**: Defines the schema for storing workout-related data, including workout type and duration.
- **ProgressModel**: Stores data related to user progress and fitness milestones.

Routes

- **UserRoutes**: Defines the API endpoints related to user operations, such as login and profile updates.
- **WorkoutRoutes**: Provides API endpoints for workout management, including scheduling and tracking workouts.

Services

- **NotificationService**: Sends notifications to users, such as reminders for scheduled workouts.

Utils

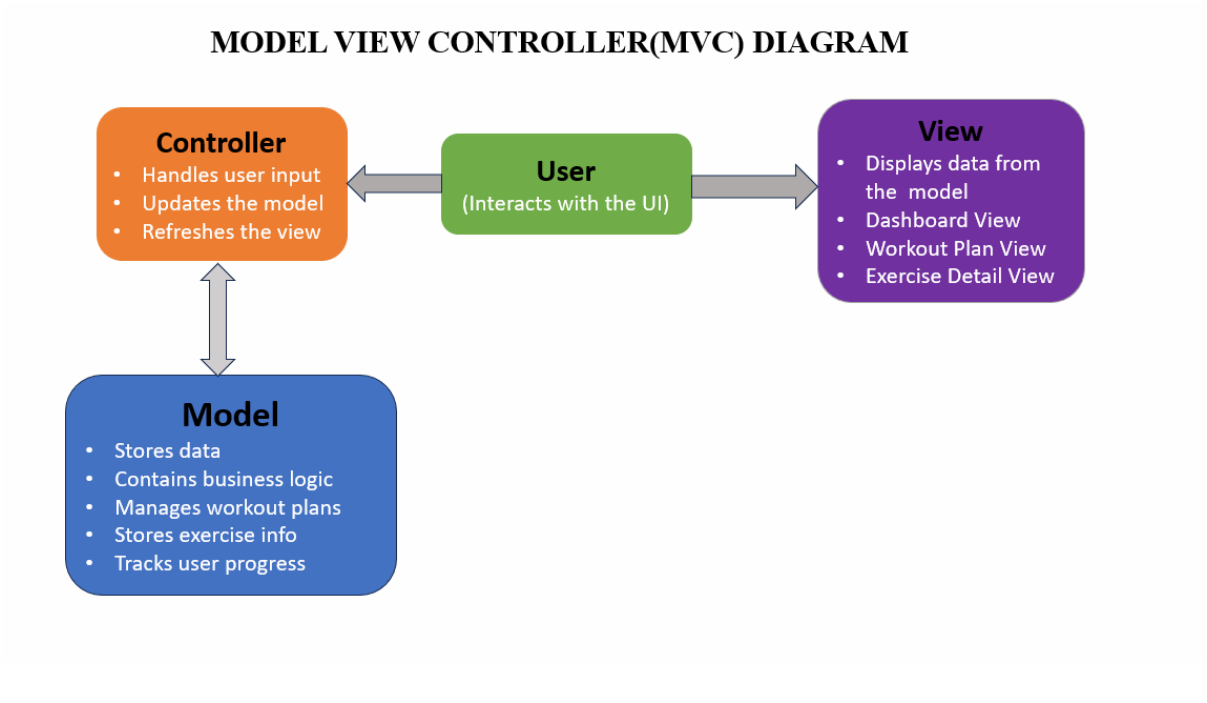
- **AuthUtils**: Provides utilities for user authentication, including token generation and verification.

Database Package

MongoDB

- **UserCollection**: Stores all user profile information, authentication details, and BMI data.
- **WorkoutsCollection**: Contains data on each workout, including type, duration, and calories burned.
- **ProgressCollection**: Tracks progress metrics for users, such as completed workouts and achievements over time.

2.1.2 Architectural Styles (MVC Diagram):



Model Section: Represents the data and business logic. In this case, it consists of:

- **WorkoutModel:** Stores workout routines, plans, and schedules.
- **ExerciseModel:** Holds details about individual exercises, like instructions and images.
- **ProgressModel:** Tracks user progress, such as completed workouts and calories burned.

View Section: Represents the user interface, where the data is displayed. The main views are:

- **Dashboard View:** It shows the work-out progress and overview of the user's work out plan in our project.
- **Workout Plan View:** This view of workout plan enables the user to design and manage workout routines.
- **Exercise Detail View:** It provides the details about an individual exercise.

Controller Section: Acts as the intermediary between the View and Model. It handles user inputs (e.g., adding a workout or searching for an exercise), updates the Model, and refreshes the View with the updated data.

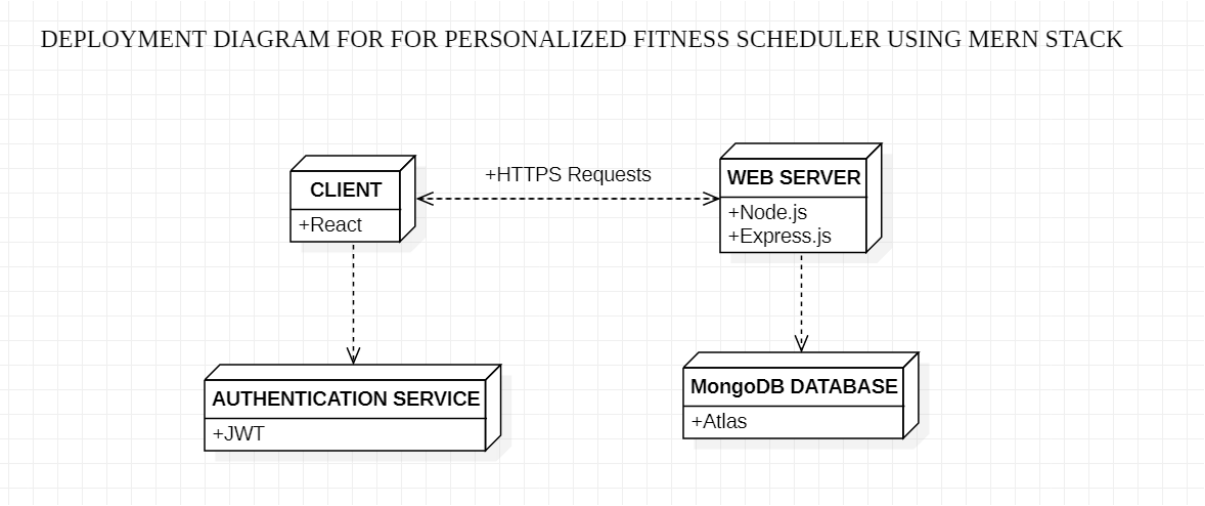
Arrows:

- **User to Controller:** The user interacts with the controller (e.g., through buttons or forms).
- **Controller to Model and View**: The controller updates the model (data) and refreshes the view with new data.

- **Model to View:** The model sends updated data back to the view for display.

2.2 Deployment Architecture

Deployment Diagram:



Client Side (Browser/Frontend)

Description: The client will interact with the application using a web browser, which will serve the React frontend.

Components:

- React Application - hosts UI components and logic for user interaction such as setting up workouts, tracking progress, and viewing profile information.
- Interaction: It sends HTTP requests to the backend server for fetching or updating information on workouts, schedules, and progress.

Web Server

- Node.js Express.js backend server handling requests from the client, processing business logic, and communicating with the MongoDB database.
- Express.js: Routing, controllers, API end-points
- API Layer: RESTful API for all users' requests, workouts, schedules, and tracking of progress
- Middleware: Authentication, Validation, Error Handling
- Interaction: It receives requests from the frontend and interacts with MongoDB to fetch or update data; responses are sent back to the client in JSON format.

Database Server (MongoDB)

Description: This is where MongoDB will store the Persisted data, for example, the user profiles, workout logs, schedules, and progress information.

Components:

- MongoDB Atlas/Local Instance: The database will have a collection for Users, Workouts, Schedules, and Progress.

- Interaction: The data storage and retrieval are communicated from the web server to MongoDB upon request by the client. In MongoDB, this data is stored in a non-relational way for flexibility and scalability.

Authentication Service

Description: Controls the authentication of users and session management.

Components:

- JWT stands for JSON Web Token: A JSON token used to protect API routes and user sessions.
- Interaction: After successfully logging in, it generates a token that is used to authenticate users' requests to the backend.

Network Details: Describe connectivity (e.g., internet, HTTP) and protocols.

2.3 Persistent Data Storage

Data Storage Overview

For the **Personalized Fitness Scheduler**, MongoDB is used as the primary database to store and manage user-related data. This NoSQL database is chosen for its flexibility in handling dynamic data and scalability, which is ideal for fitness tracking and personalization. Key collections include:

- **Users**: Stores user profile information and authentication details.
- **Workouts**: Contains individual workout logs with details such as type, duration, and intensity.
- **Schedules**: Holds user-specific workout schedules, including dates, times, and assigned workouts.
- **Progress**: Tracks fitness progress over time, helping users visualize improvements and reach goals.

Database Schema

- **Users Collection**
  - **userId** (ObjectId): Unique identifier for each user.
  - **name** (String): User's full name.
  - **email** (String): User’s email for login and notifications.
  - **passwordHash** (String): Hashed password for secure login.
  - **bmr** (Number): User's Body Mass Index, calculated based on weight and height.
  - **createdAt** (Date): Date the user account was created.
- **Workouts Collection**
  - **workoutId** (ObjectId): Unique identifier for each workout entry.
  - **userId** (ObjectId): Reference to the user performing the workout.
  - **type** (String): Type of workout (e.g., cardio, strength).
  - **duration** (Number): Duration of workout in minutes.
  - **intensity** (String): Workout intensity (e.g., low, medium, high).
  - **date** (Date): Date of the workout session.
- **Schedules Collection**
  - **scheduleId** (ObjectId): Unique identifier for each workout schedule.
  - **userId** (ObjectId): Reference to the user for whom the schedule is created.

- **workoutId** (ObjectId): Reference to a specific workout from the Workouts collection.
- **scheduledDate** (Date): Date and time for the scheduled workout.
- **isCompleted** (Boolean): Status indicating if the workout was completed.

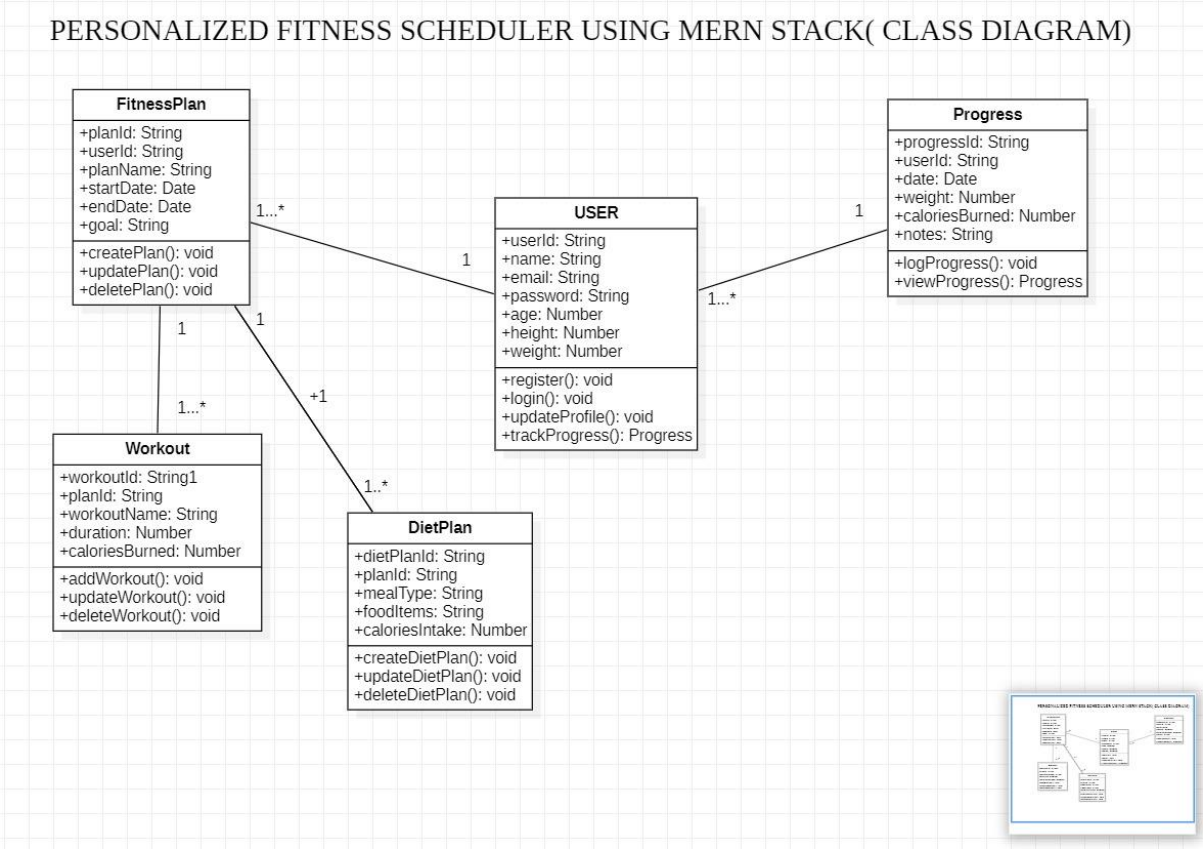
- **Progress Collection**
  - **progressId** (ObjectId): Unique identifier for each progress entry.
  - **userId** (ObjectId): Reference to the user tracking their progress.
  - **workoutId** (ObjectId): Reference to a workout session in the Workouts collection.
  - **caloriesBurned** (Number): Approximate calories burned during the workout.
  - **distance** (Number, optional): Distance covered, if applicable (e.g., running).
  - **timestamp** (Date): Date and time of the progress entry.

2.4 Global Control Flow

- **Control Flow Model**: This application uses an **event-driven model**, where user interactions trigger specific events, such as scheduling a workout or updating excercises.
- **Concurrency**: Although the website is largely single-threaded, some asynchronous handling is in place for API requests to ensure smooth user interactions without blocking.
- **Timer Dependencies**: None explicitly, but front-end timers may be used for session management or reminders.

3. Detailed System Design

3.1 Static View (Class Diagrams)

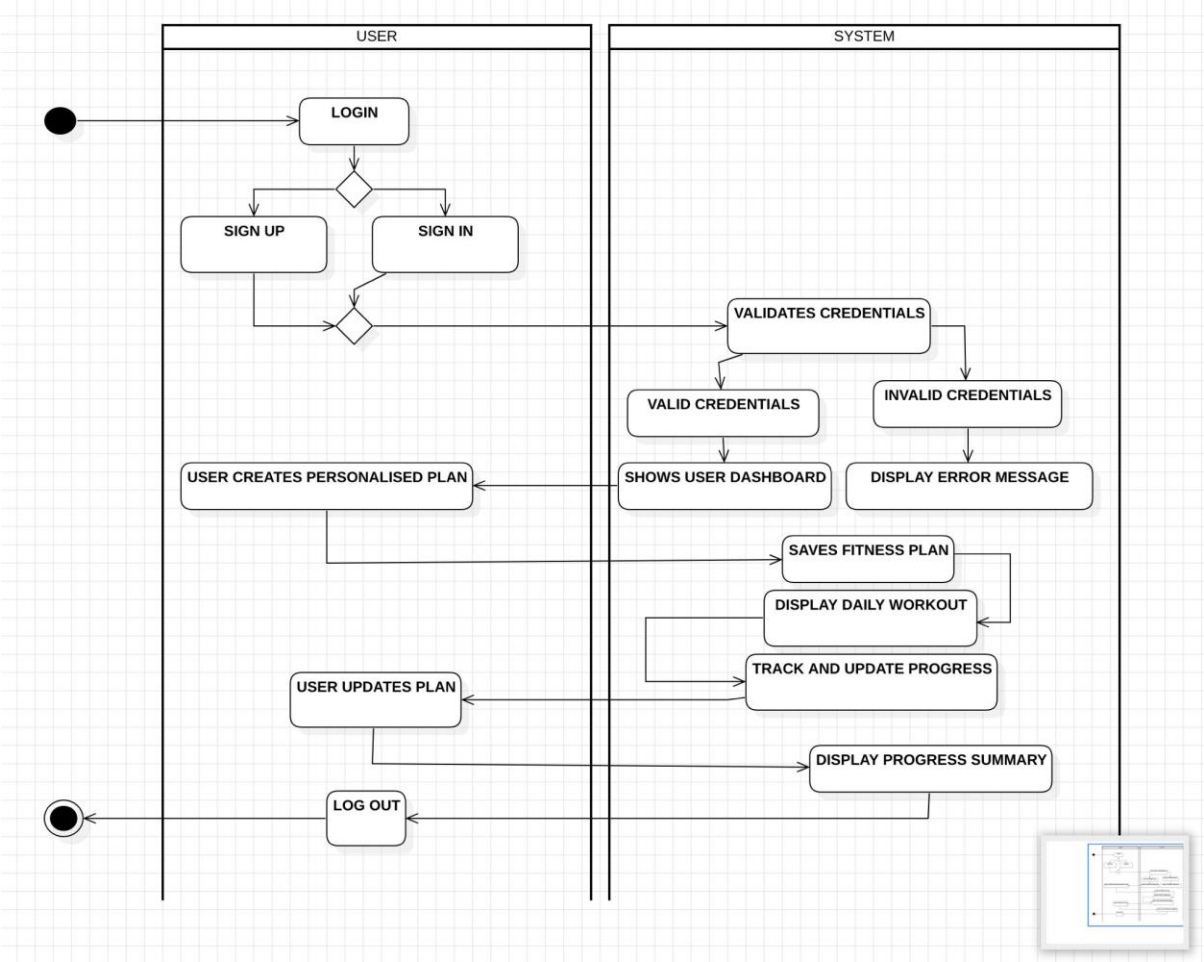


- **FitnessPlan**
  - planId: Unique identifier for the fitness plan.
  - userId: Identifier linking the plan to a user.
  - planName: Name of the fitness plan.
  - startDate: Start date of the fitness plan.
  - endDate: End date of the fitness plan.
  - goal: The goal of the fitness plan.
  - createPlan(): Method to create a new fitness plan.
  - updatePlan(): Method to update the fitness plan.
  - deletePlan(): Method to delete the fitness plan.
- **USER**
  - userId: Unique identifier for the user.
  - name: Name of the user.
  - email: Email address of the user.
  - password: User's password.
  - age: Age of the user.
  - height: Height of the user.
  - weight: Weight of the user.
  - register(): Method to register a new user.
  - login(): Method for user login.
  - updateProfile(): Method to update the user profile.
  - trackProgress(): Method to track the user's progress.
- **Workout**
  - workoutId: Unique identifier for the workout.
  - planId: Identifier linking the workout to a fitness plan.
  - workoutName: Name of the workout.
  - duration: Duration of the workout.
  - caloriesBurned: Calories burned during the workout.
  - addWorkout(): Method to add a new workout.
  - updateWorkout(): Method to update the workout details.
  - deleteWorkout(): Method to delete a workout.
- **DietPlan**
  - dietPlanId: Unique identifier for the diet plan.
  - planId: Identifier linking the diet plan to a fitness plan.
  - mealType: The kind of meal the user takes for breakfast, lunch, dinner.
  - foodItems: The list of food items in the diet plan of the user.
  - caloriesIntake: Number of calories intake for the diet plan.
  - createDietPlan(): Method to create a new diet plan.
  - updateDietPlan(): Method to update the diet plan.
  - deleteDietPlan(): Method to delete the diet plan.
- **Progress**
  - progressId: Unique identifier for the progress entry.
  - userId: Identifier linking the progress entry to a user.
  - date: Date of the progress entry.
  - weight: User's weight at the time of the entry.
  - caloriesBurned: Calories burned as recorded in the progress entry.
  - notes: Additional notes about the progress.
  - logProgress(): Method to log the user's progress.

- viewProgress(): Method to view the user's progress.

3.2 Dynamic View (Sequence Diagrams)

- **Sequence Diagrams**:



- **LOGIN:** The starting point where the user can either sign up or sign in.
- **SIGN UP:** Process for new users to create an account.
- **SIGN IN:** Process for existing users to access their accounts.
- **VALIDATES CREDENTIALS:** The system is going to check the provided login information.
- **SHOWS USER DASHBOARD:** System displays the main interface for the user.
- **USER CREATES PERSONALISED PLAN:** User sets up a custom fitness plan.
- **SAVES FITNESS PLAN:** System saves the user's custom fitness plan.
- **DISPLAY DAILY WORKOUT:** System shows the user's daily exercise routine.
- **TRACK AND UPDATE PROGRESS:** System monitors and logs user's fitness progress.
- **DISPLAY PROGRESS SUMMARY:** System provides a summary of the user's progress.
- **USER UPDATES PLAN:** User modifies their existing fitness plan.
- **LOG OUT:** User exits the application.

4. Design Rationale

Rationale for Choices

- **Architecture**: MERN stack was selected for its cohesiveness and performance. The flexibility of JavaScript from front-end to back-end aids in developing a uniform, real-time experience.
- **Storage Solution**: MongoDB was chosen over SQL for its flexibility with JSON-like documents, which easily adapts to fitness data structures that may evolve over time.
- **Design Patterns**: The Factory and Singleton patterns are applied to simplify future changes and optimize resource management, respectively.

5. Testing and Sprints

Test Plans

- **Core Test Cases**:
    - **User Registration Test:** Verifying the creation of a new user profile with correct data validation.
    - **Workout Scheduling Test:** Ensuring a workout is correctly scheduled and saved.
    - **Profile Update Test:** Checking if updates to user metrics are stored and displayed accurately.
    - **Progress Tracking Test:** Confirming accurate logging and retrieval of workout history.

Sprints Overview

- **Sprint Goals and Reviews**:
    - **Sprint 1(Completed)**

      Goal: The aim is to set up the development environment and start designing UI/UX.

      Review: Development tools were pinpointed, including Node.js, React, and MongoDB. Some wireframing of the Fitness Scheduler interface was done. Since the detailed design of intuitive interfaces takes some time, some more detailed UI design activities were scheduled in Sprint 2.
    - **Sprint 2(Completed)**

      Goal: To develop the backend infrastructure and implement the Workout Plan feature.

      Review: Designed Node.js backend with MongoDB database, created APIs for user registration and workout log, and developed the Workout Plan feature. Some of the tasks which needed integration with the front end and making the UI responsive were moved to Sprint 3 for refinement.
    - **Sprint 3(In Progress)**

      Goal: Refine front-end, develop all the feature, and finalize back-end integration.

Review: The progress made included the development of UI optimization for various devices, a Progress Tracker for visualization of fitness progress, and completing the integration between the frontend and backend components to support seamless user experiences.